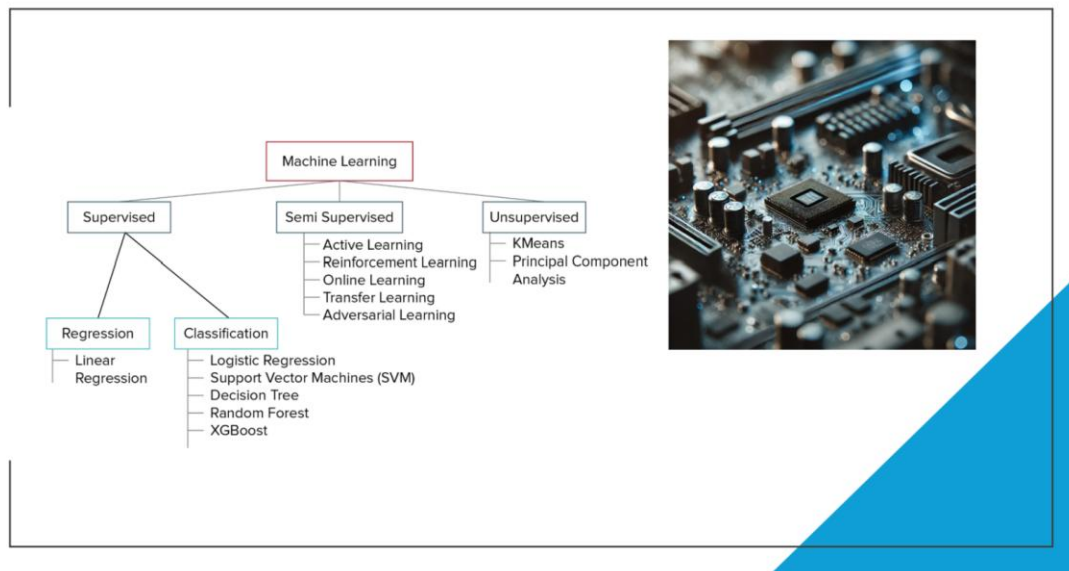


Module 2 – Dr. Amanda’s Office Hours Algorithm Deeper Dive Cheat Sheet



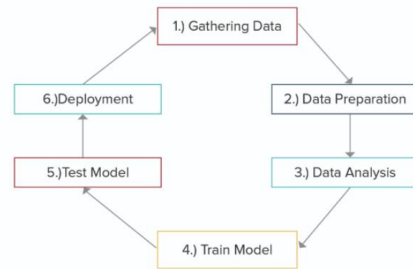
Three primary categories of machine learning (ML): Supervised Learning, Semi-Supervised Learning, and Unsupervised Learning.

1. **Supervised Learning** uses labeled data to train algorithms, which can be divided into two types:
 - **Regression:** Predicts continuous outcomes (e.g., house prices).
 - **Classification:** Predicts discrete categories (e.g., spam detection).
2. **Semi-Supervised Learning** combines a small amount of labeled data with a large amount of unlabeled data to make decisions with minimal supervision, using techniques like Active Learning and Reinforcement Learning.
3. **Unsupervised Learning** works with unlabeled data, finding patterns such as:
 - **Clustering:** Groups similar data (e.g., customer behavior).
 - **Dimensionality Reduction:** Simplifies data while retaining key features (e.g., PCA).

Each of these methods serves different applications, from predicting outcomes to classifying or organizing data.



ML Lifecycle

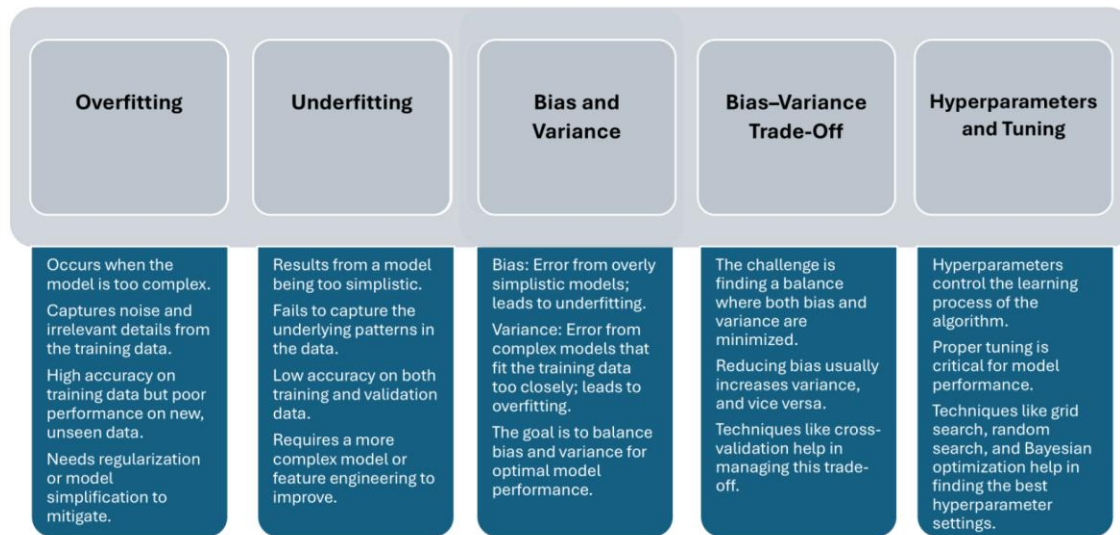


The image illustrates the Machine Learning (ML) Lifecycle, which includes the following key stages:

1. Data Gathering: Collecting relevant data for analysis.
2. Data Preparation: Cleaning and organizing the data for processing.
3. Data Analysis: Exploring and understanding the data to inform model development.
4. Model Training: Building and training the machine learning model using the prepared data.
5. Model Testing: Evaluating the model's performance to ensure accuracy.
6. Deployment: Implementing the model in a real-world environment.

This lifecycle is a continuous process that iteratively improves the machine learning model for optimal results.

Basic Concepts and Terminologies



Grid search, random search, and Bayesian optimization are methods used to optimize hyperparameters, which are critical settings in machine learning algorithms that can significantly affect a model's performance.

1. Grid Search:

- Grid search is an exhaustive search method where all possible combinations of a set of hyperparameters are evaluated.
- For example, if you want to tune two hyperparameters, such as the number of trees in a random forest and the maximum depth of the trees, grid search will try every possible combination of these parameters within specified ranges.
- While thorough, grid search can be computationally expensive, especially with a large number of hyperparameters or a wide range of values.

2. Random Search:

- Random search, as the name suggests, involves randomly sampling combinations of hyperparameters within specified ranges.
- Unlike grid search, it doesn't try every possible combination but rather explores the hyperparameter space more efficiently by randomly selecting combinations.
- This approach can be faster than grid search and often finds good solutions with less computational cost, particularly in cases with many hyperparameters.

3. Bayesian Optimization:

- Bayesian optimization is a more sophisticated method that uses probability models to guide the search for the best hyperparameters.

- Instead of trying random or grid-like combinations, Bayesian optimization builds a model of the performance of the hyperparameters and uses this model to predict which combinations might be the most promising.
- It iteratively evaluates new hyperparameter combinations based on the model's predictions, focusing on areas of the hyperparameter space that are expected to yield the best results. This method is generally more efficient than grid or random search and can often find the best hyperparameters with fewer evaluations.

Each of these techniques helps in finding the most effective hyperparameters, leading to improved model accuracy and generalization without the need for extensive manual tuning.

Types of Machine Learning Models

Parametric Models		Non - Parametric Models		Discriminative Models		Generative Models		Generative Adversarial Networks (GANs)	
Explanation	Use Cases	Explanation	Use Cases	Explanation	Use Cases	Explanation	Use Cases	Explanation	Use Cases
These models have a set number of parameters that don't change with the amount of data. They assume a specific form for the function mapping inputs to outputs. Example Models: Linear Regression, Logistic Regression, OLS	Predicting Housing Prices: Linear regression can predict house prices based on fixed factors like square footage and number of bedrooms. Email Spam Detection: Logistic regression helps classify emails as spam or not based on a fixed set of features.	Flexible Model Complexity: These models can adapt to the data more flexibly as they don't assume a fixed number of parameters. Example Models: Decision Trees, k-Nearest Neighbors (k-NN), Random Forests.	Customer Segmentation: k-NN can classify new customers into existing segments based on purchasing patterns. Medical Diagnosis: Random Forests can help predict diseases based on patient history and symptoms.	Focus on directly mapping inputs to outputs, predicting outcomes based on input features, and are often referred to as forward models. Ex. Neural networks, including deep, convolutional, and recurrent	Image Classification: Neural networks are used to classify images in apps like Google Photos. Speech Recognition: Deep Neural Networks are used in applications like virtual assistants.	Learn Data Distribution: These models focus on modeling class distributions and generating new data points. Example Models: Bayesian Models for probabilistic reasoning under uncertainty; Markov Chain Monte Carlo (MCMC) for approximating parameter distributions in complex models.	Medical Diagnosis: Bayesian models estimate disease risk based on patient symptoms and test results. Climate Modeling: MCMC methods simulate and predict future climate scenarios by estimating environmental variables.	Adversarial Learning: Combines a generative model with a discriminative model in a game-theoretic scenario to generate realistic data. Example Models: Standard GAN, StyleGAN, CycleGAN.	Synthetic Data Generation: GANs are used to generate synthetic data for training machine learning models, particularly when real data is scarce or sensitive, such as in healthcare or autonomous driving simulations. Deepfake Creation: GANs are behind many deepfake technologies, generating realistic faces or voices.

Linear Classification

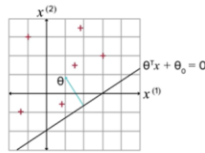
- Consider an input (vector) feature space x
- Assume we have two classes:
 $y \in \text{"Positive" (+) or "Negative" (-)}$

- A linear classifier: A linear separator with model parameters

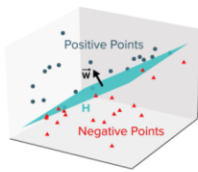
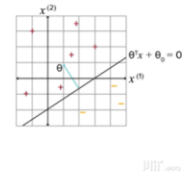
$$\theta^T x + \theta_0 = 0$$

$$\begin{bmatrix} -2 & 3 \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \end{bmatrix} + 6 = 0$$

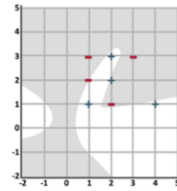
- Inference: Decide "+" if $\theta^T x + \theta_0 > 0$ else "-"



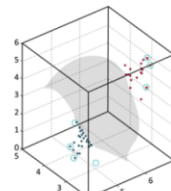
- Training occurs with some algorithm that uses some set of training data to find the coefficients θ^T and θ_0



Transformation
 $f(x) = x_1, x_2$
Enables a simple linear
classifier for the data



A higher order $f(x)$ enables a
linear classifier for the data



A higher order $f(x)$ enables a complex
linear classifier for the data

Linear Classification, diagrams explained:

Top Left Diagram (Linear Classifier Basics):

This diagram shows a basic setup where you have two types of data points (like two different species of animals, for example). The data points are marked with "+" and "-", and they're scattered on a grid. A straight line is drawn between them.

The line is a "decision boundary," which means it's the line the computer uses to decide which side a new data point belongs to. If it falls on one side of the line, it's classified as one type (like "+" or the first species), and if it falls on the other side, it's classified as the other type (like "-" or the second species).

To put this into perspective, imagine you're sorting apples and oranges on a table, and you draw a line to keep them apart. The line helps you quickly figure out if any new fruit on the table is an apple or an orange based on which side of the line it falls.

Top Right Diagram (Training the Linear Classifier):

This diagram shows a similar setup, but it's focusing on how the line (or boundary) is found. It's like adjusting the line to best separate the "+" and "-" points based on the training data you have.

The process of adjusting this line to separate the data points correctly is called "training." During training, the line is moved around until it fits the data as well as possible.

Think of this like fine-tuning that line on the table until all the apples are on one side and all the oranges are on the other. The goal is to get it just right so you can always sort them perfectly.

Bottom Left Diagram (Linear Classifier with Transformation):

In this diagram, the data points are more mixed up, and a straight line might not work well to separate them. The idea here is that you can transform the data into a new space where it is easier to separate with a line.

Sometimes, data is too complex to be separated by a simple line in its original form. So, we can transform it into a higher dimension (like going from 2D to 3D), where a line can then separate the points.

Going back to our fruit example, if you can't easily separate your apples and oranges on a flat table, imagine you could lift the table into 3D space. Maybe now, they can be separated with a simple sheet of paper (a plane) instead of just a line.

Bottom Middle Diagram (Non-Linear Decision Boundary):

This diagram shows that instead of a straight line, a curved line (or boundary) is drawn to separate the two types of data points.

When a straight line isn't enough to separate the data, we use a curved line. This approach is still based on a linear classifier but applied in a more complex way, allowing it to handle trickier data distributions.

Imagine that your apples and oranges are scattered in such a way that a straight line won't work. Instead, you draw a curvy line to make sure all the apples are on one side and all the oranges are on the other.

Bottom Right Diagram (3D Visualization of Complex Classification):

This diagram shows a more advanced scenario where the data is represented in three dimensions. The decision boundary is now a plane (a flat sheet in 3D space) that separates the data points.

When dealing with complex data, we can think in three dimensions instead of two. Here, the decision boundary is like a sheet of paper that separates the data into different groups based on their position in this 3D space.

In our final fruit example, imagine you're now using a flat piece of cardboard instead of a line to separate your apples and oranges, but now they're floating in a 3D space (like on a shelf with different levels). The cardboard slices through this space to separate the two groups.

Linear Classification

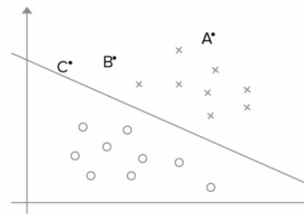
Consider the simplest linear classifier in two dimensions: a simple two-dimensional case with two features x_1 and x_2 .

The decision boundary is a line defined by:

$$w_0 + w_1x_1 + w_2x_2 = 0$$

Where w_0 , w_1 , w_2 are the y coefficient and coefficients of x_1 and x_2 , respectively. The equation of the straight line represents the linear decision boundary. Using the vector notation, the decision boundary line in this case is given by:

$$w_0 + w^T X = 0$$



College Admission



Credit Risk Assessment



Customer Churn



1. First Diagram (Linear Classifier Equation):

- The diagram presents the mathematical equation that defines the **decision boundary**—the line that separates two classes in a 2D space (e.g., apples vs. oranges).
- **w_0 (bias/intercept):** This is a constant term in the equation that shifts the line up or down on the graph, affecting where the line crosses the axes.
- **w_1 and w_2 (coefficients/weights):** These control the slope and position of the line based on the values of features x_1 and x_2 . For example:
 - x_1 could represent the weight of a fruit.
 - x_2 could represent its color.
- **Equation:** When put together, these weights and features form a line, separating data points into two categories (e.g., "+" for apples and "-" for oranges).
- **Vector Notation:** The equation can also be written in a more compact form using vectors, which organizes all the weights (w) and features (x) neatly. This is useful when dealing with many features.

Concept:

- The equation helps create a **line** in the data space that acts as a boundary between two classes. The classifier will use this line to determine the class of new data points based on which side of the line they fall on.

Further Context:

- Imagine you're trying to sort apples and oranges based on two features—say weight and color. The coefficients in the equation tell you how important each feature is for making the decision. The equation itself is like drawing a line on a graph, where one side represents apples, and the other side represents oranges.

2. Second Diagram (Graphical Representation of Linear Classifier):

- The second diagram provides a **visual representation** of the equation from the first diagram. It shows a **graph** where the decision boundary (a straight line) separates data points into two categories.
- **Data Points (A, B, C):**
 - **A:** Clearly falls on one side of the decision boundary (e.g., an apple).
 - **C:** Falls on the opposite side (e.g., an orange).
 - **B:** A borderline case, close to the line, and harder to classify definitively.

Concept:

- The **decision boundary** (the line) separates the two classes (e.g., apples and oranges). The classification of any new data point depends on which side of the line it lands on.

Further Context:

- Think of this graph like a **map** where you're sorting items (apples and oranges). The line helps you classify new data points. For instance, points like A are clearly on the apple side, and points like C are on the orange side. B is tricky because it's close to the line, so the classifier might struggle to categorize it perfectly.

1. College Admissions Prediction:

Application of Concepts:

- **Features:** The model might use features like:
 - x_1 (X sub one) could represent the student's SAT score.
 - x_2 (X sub two) could represent the student's high school GPA.
 - x_3 (X sub three) could represent the number of extracurricular activities.
- **Weights:** The model learns weights w_1 (W sub one), w_2 (W sub two), and w_3 (W sub three) that determine how important these features are in predicting whether a student will be admitted.
- **Decision Boundary:** The equation $w_0 + w_1x_1 + w_2x_2 + w_3x_3 = 0$ (W naught plus W sub one X sub one plus W sub two X sub two plus W sub three X sub three equals zero) defines a line that separates students into "Admitted" and "Not Admitted" categories.

Real-World Example:

- **College Admissions:** Colleges might use linear classifiers to predict whether to admit a student. Based on where a student's features fall in relation to the decision boundary, the system decides whether the student is likely to be admitted or not.

2. Credit Risk Assessment:

Application of Concepts:

- **Features:** The model might consider:
 - x_1 (X sub one) as the applicant's credit score.
 - x_2 (X sub two) as the applicant's annual income.
 - x_3 (X sub three) could represent the number of existing loans or debts.
- **Weights:** The model learns weights w_1 (W sub one), w_2 (W sub two), and w_3 (W sub three) to balance these features and assess risk.
- **Decision Boundary:** The linear classifier uses an equation like $w_0 + w_1x_1 + w_2x_2 + w_3x_3 = 0$ to classify applicants as "High Risk" or "Low Risk."

Real-World Example:

- **Loan Approvals:** Banks and financial institutions use such models to quickly evaluate whether a loan applicant is likely to repay a loan or default. This helps in making informed decisions on whether to approve or deny a loan application.

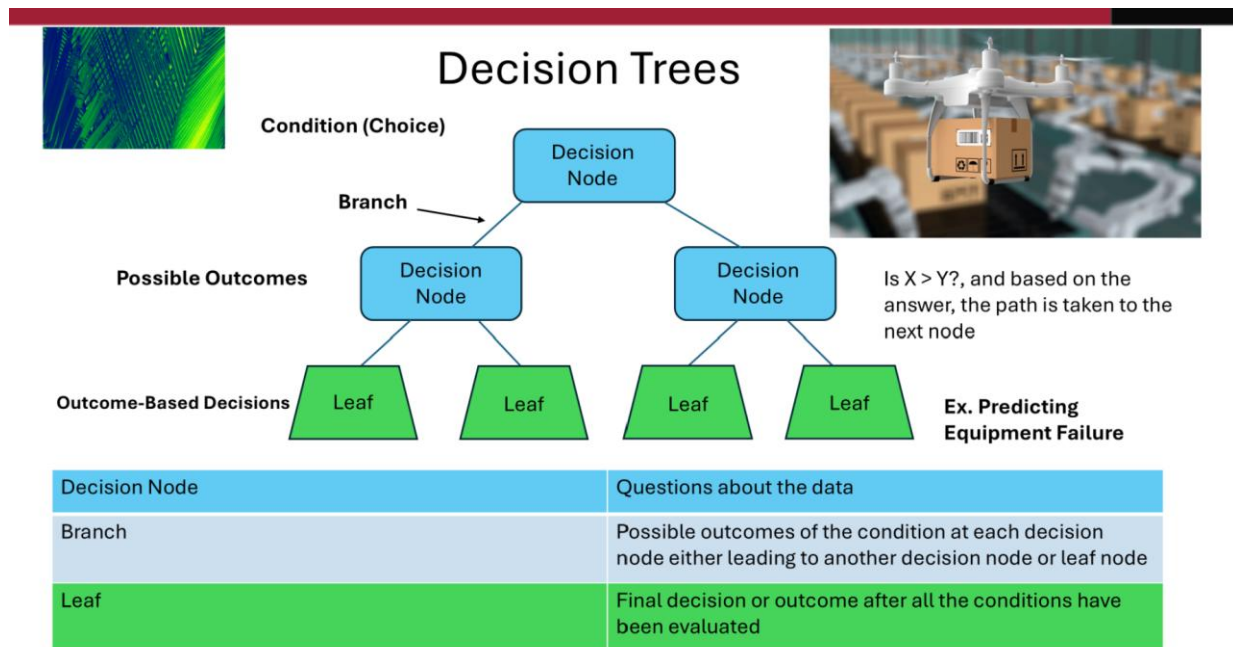
3. Customer Churn Prediction:

Application of Concepts:

- **Features:** The model might analyze:
 - x_1 (X sub one) as the number of times a customer has contacted customer support in the last month.
 - x_2 (X sub two) as the total amount the customer has spent in the last year.
 - x_3 (X sub three) could be the duration of the customer's subscription (how long they've been with the company).
- **Weights:** The model adjusts the weights w_1 (W sub one), w_2 (W sub two), and w_3 (W sub three) to find the best decision boundary.
- **Decision Boundary:** The decision boundary helps classify customers as "Likely to Churn" or "Likely to Stay," based on the equation $w_0 + w_1x_1 + w_2x_2 + w_3x_3 = 0$.

Real-World Example:

- **Retention Strategies:** Companies use these models to identify which customers are at risk of leaving (churning). By predicting churn, companies can take proactive steps, like offering discounts or personalized offers, to retain those customers.



This diagram captures the key elements of a decision tree and includes an explanation of what each part represents.

Decision Node (Blue Boxes):

- Their function is representing a point where a decision is made based on a specific condition or feature in the dataset. For example, a decision node might represent the question "Is $X > Y$ ". Depending on the answer (yes or no), the path leads to another decision node or a leaf node. The condition at each decision node is a critical part of how the tree branches out.

Branch (Lines Connecting Nodes):

- Branches represent the possible outcomes of a decision made at a node. Each branch leads to either another decision node or a final decision (leaf). This is where the tree diverges based on the results of the decision.

Leaf (Green Boxes):

- Leaves are the final decision or classification made by the tree after all the conditions have been evaluated. This could be an output like "Class A" or "Class B" in a classification problem, or a numerical value in regression.

A good example of using decision trees in a different context is predicting equipment failure in manufacturing. In this scenario, a decision tree can analyze data from machinery, such as temperature readings, vibration levels, and operational hours, to predict when a piece of equipment is likely to fail. The tree's structure helps identify patterns and conditions that are indicative of potential breakdowns. This allows maintenance teams to take proactive measures, like scheduling repairs before a failure occurs, thereby reducing downtime and maintenance costs.

Decision Tree creation:

<https://www.smartdraw.com/decision-tree/>

smartdraw Product Solutions Resources Enterprise Pricing My Account Contact Sales Launch App

Decision Tree

Generate decision trees quickly from data or a built-in template

Create Your Decision Tree

smartdraw Product Solutions Resources Enterprise Pricing My Account Contact Sales Launch App

Seven Tips for Creating a Decision Tree

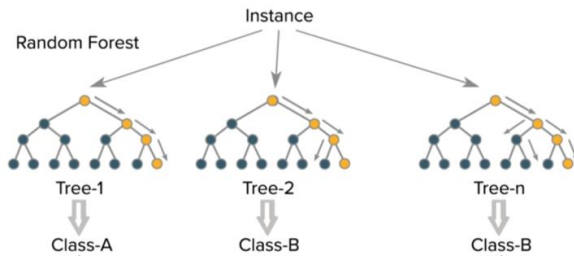
Here are some best practice tips for creating a decision tree diagram:

- **Start the tree.** Draw a rectangle near the left edge of the page to represent the first node. In this rectangle, write the first question, main idea, or criterion that will lead to a decision.
- **Add branches.** For every possible alternative draw a separate line that begins at the node and moves away toward the right of the page. Using a loan approval process as an example, the first node may have been "Income", and the associated branches might be <\$50K, \$51K - \$100K, >\$101K.
- **Add leaves.** The bulk of the decision tree will be leaf nodes. At the end of each branch add a leaf node. Fill each of these leaf nodes with another question or criterion.
- **Add more branches.** Repeat the process of adding a branch for each possible alternative leading from a leaf. Label each branch just as before.
- **Complete the decision tree.** Continue adding leaves and branches until every question or criterion has been resolved and an outcome has been reached.
- **Terminate a branch.** Continue adding leaves and branches until every question or criterion has been resolved and an outcome has been reached.
- **Verify accuracy.** Consult with all stakeholders to verify accuracy.

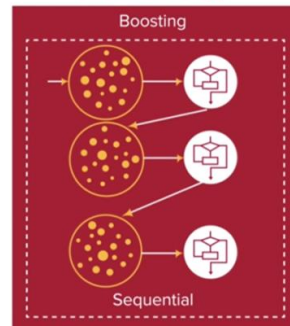
[Back to top](#)

<https://www.smartdraw.com/decision-tree/>

Random Forest Machine Learning Model



Boosting



Bagging and Boosting? Quantdare, April 20, 2016.
[re:en-bagging-and-boosting/](#)

Ex. Inventory
Optimization



This diagram illustrates how a **Random Forest** model works. **Random Forests** are an ensemble learning method that builds multiple decision trees on random subsets of the data and features, then aggregates their predictions through majority voting (classification) or averaging (regression). This approach increases model robustness and reduces overfitting.

- **Instance:** An instance (or input) is the data point that needs to be classified or predicted.
- **Trees:** The Random Forest consists of multiple decision trees (Tree-1, Tree-2, Tree-n). Each tree is trained on a different random subset of the data and features.
- **Class Predictions:** Each tree makes its own prediction about the class of the instance. For example, Tree-1 predicts Class-A, while Tree-2 and Tree-n predict Class-B.
- **Majority Voting:** The final class prediction is determined by majority voting, where the class with the most votes from the individual trees becomes the final output.

Boosting, on the other hand, sequentially builds trees, where each tree focuses on correcting the errors of the previous ones, often leading to higher accuracy but with a risk of overfitting.

A relevant problem in retail where Random Forest can be applied is inventory optimization. In this scenario, Random Forests can be used to predict the optimal stock levels for various products across multiple store locations. The model can analyze factors such as historical sales data, seasonal trends, supplier lead times, and storage costs. Each tree in the Random Forest might focus on different aspects, such as regional sales patterns or the impact of holidays on inventory needs. By combining the results from all trees, the Random Forest model can provide recommendations on how much stock to hold at each location to minimize both stockouts and excess inventory.

This approach is beneficial for retail chains looking to optimize their supply chain, reduce holding costs, and improve operational efficiency without directly involving customer behavior in the analysis.

Bayesian Methods



The probability of event A given that event B has occurred is equal to the probability of event B given that event A has occurred, multiplied by the probability of event A, and then divided by the probability of event B.

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)}$$

Where:

$P(A|B)$ is the probability of event A, given that event B has occurred. → Given that it's cloudy (event B), what's the chance it's going to rain (event A)?

$P(B|A)$ is the probability of event B given that event A has occurred. → The chance of seeing clouds if we know it's going to rain

$P(A)$ and $P(B)$ are the probabilities of events A and B respectively. → The probability of A happening before you even consider B. So, before you look at the clouds, how likely is rain based on your general knowledge (like weather patterns)? The probability of B happening on its own - how likely it is that you would see clouds, regardless of whether it's going to rain or not.

- $P(A|B)$ This is what we're trying to figure out—the probability of event A happening given that B has occurred. It's like saying, "Given that it's cloudy (event B), what's the chance it's going to rain (event A)?"
- $P(B|A)$ This is the likelihood, which we talked about earlier. It's the probability of seeing event B if we know A has happened. In our weather example, this could be the chance of seeing clouds if we know it's going to rain.
- $P(A)$ This is your prior knowledge—the probability of A happening before you even consider B. So, before you look at the clouds, how likely is rain based on your general knowledge (like weather patterns)?
- $P(B)$ This is the probability of B happening on its own. It's just how likely it is that you would see clouds, regardless of whether it's going to rain or not.

Bayes' Theorem essentially combines what you know about the likelihood ($P(B|A)$) and your prior knowledge ($P(A)$) to update your belief about $P(A|B)$ the probability of A happening given B.

Real-Life Examples Where Bayesian Methods Are Useful:

1. Medical Diagnosis:

- Imagine you're a doctor trying to determine if a patient has a certain disease. (A) could be the event that the patient has the disease, and (B) could be the event that the patient tested positive. Using Bayes' Theorem, you can update the probability that the patient actually has the disease based on the test result and how reliable the test is (i.e., the likelihood).

2. Spam Filtering in Email:

- Here, (A) could be the event that an email is spam, and (B) could be certain words or phrases appearing in the email. Bayesian methods help to update the probability

that an email is spam based on the words it contains and how likely those words are to appear in spam emails versus regular emails.

3. Predicting Sports Outcomes:

- Say you're trying to predict whether a certain team will win a game (event A). You might consider the probability of them winning given that they've won their last few games (event B). Using Bayes' Theorem, you can update your belief about their chances based on their recent performance.

4. Weather Forecasting:

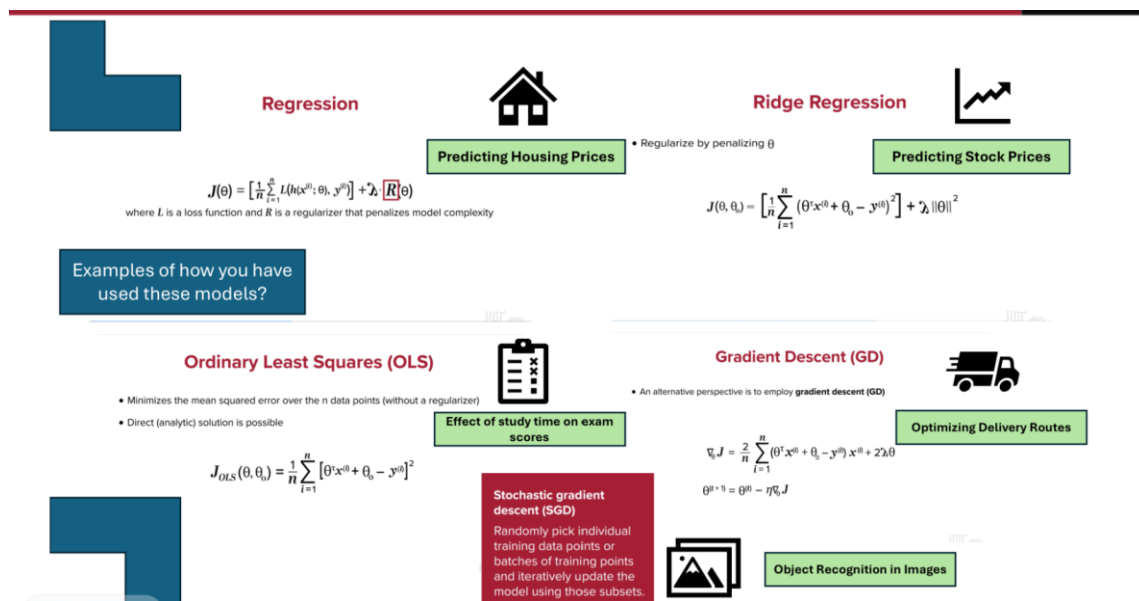
- We touched on this earlier, but in a more technical sense: (A) could be the probability of rain, and (B) could be seeing dark clouds. Based on how often clouds lead to rain in the past, you can update the forecast for rain today using Bayes' Theorem.

5. Finance and Stock Market Predictions:

- In finance, Bayesian methods can be used to predict stock prices. (A) might be the event that a stock's price will go up, and (B) could be recent news about the company. By updating your prior belief about the stock's performance with the new information, you can make more informed investment decisions.

6. Machine Learning and AI:

- Bayesian methods are heavily used in machine learning for things like classification problems. For example, if you're building a system to recognize cats in images, (A) could be the event that a certain image contains a cat, and (B) could be specific features in the image (like the shape of ears). As the system sees more images and learns from them, it updates its belief about whether a new image contains a cat using Bayesian methods.



Regression: Regression is a statistical technique used to model the relationship between a dependent variable and one or more independent variables. It aims to predict the dependent variable based on independent variable values. To avoid overfitting, regularization can be applied, which penalizes overly complex models.

Example: Predicting housing prices based on various factors such as the size of the house, number of bedrooms, location, etc. Regularization would help prevent the model from becoming too complex, which could lead to poor generalization to new data.

Ridge Regression: Ridge Regression introduces a penalty based on the square of the coefficient magnitudes to deal with multicollinearity. Multicollinearity happens when two or more factors in a model are so closely related that it becomes hard to figure out which one is really affecting the outcome, making the model's predictions less reliable. It shrinks the coefficients, improving the model's stability and reliability.

Example: Suppose you're predicting a stock's price based on multiple economic indicators. If some indicators are correlated, Ridge Regression would help manage the collinearity, ensuring the model's predictions remain stable.

Ordinary Least Squares (OLS): OLS is a linear regression method that minimizes the sum of the squared differences between observed and predicted values. It is a basic regression technique without regularization.

Example: Estimating the effect of hours studied on exam scores. OLS would find the line that best fits the data by minimizing the sum of the squared errors between the predicted scores and the actual scores.

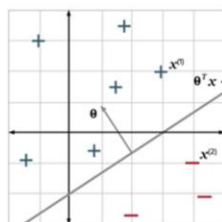
Gradient Descent (GD): Gradient Descent is an iterative algorithm used to minimize the cost function by adjusting parameters in the direction that reduces the overall cost most efficiently.

Example: In logistics and supply chain management, companies often face the challenge of optimizing delivery routes for a fleet of vehicles. By applying Gradient Descent, the algorithm can iteratively adjust the sequence of deliveries and the path taken between stops to minimize the total distance traveled or the total time spent on deliveries. Over time, Gradient Descent fine-tunes the route to achieve the most efficient delivery plan, which can result in reduced fuel consumption, lower operational costs, and faster delivery times.

Stochastic Gradient Descent (SGD): A variant of Gradient Descent where the model parameters are updated for each training example, rather than for the entire dataset as in regular Gradient Descent. This makes it faster and more suitable for large datasets.

Example: Training a deep learning model to recognize objects in images. SGD would update the model parameters after each image, making the training process faster and allowing it to handle large datasets more effectively.

Logistic Regression



- Consider an input (vector) feature space x
- Envision output y as the *probability* of membership in the class with label $y=1$ vs. in the class with label $y=0$
- Linear function (as in linear regression) for input x :

$$Z = \theta^T x + \theta_0$$

- Add a nonlinearity $\sigma(z)$ that "squashes" z to the range $[0, 1]$ indicating guess g as probability of class membership:

$$g = \sigma(z) = \frac{1}{1 + e^{-z}}$$

Wikipedia

- Define the negative *log-likelihood* loss function between a guess $g^{(i)}$ for data point and the actual class $y^{(i)}$:
 $(g^{(i)}, y^{(i)}) = y^{(i)} \log g^{(i)} + (1 - y^{(i)}) \log(1 - g^{(i)})$
- Optimize to minimize this error (to penalize large θ):

$$J(\theta, \theta_0) = \left(\frac{1}{n} \sum_{i=1}^n L_{\text{log}}(g^{(i)}, y^{(i)}) \right) + \lambda \|\theta\|$$

Inference: typically decide class = 1 if $g > 0.5$
else class = 0

Real-World Example: Email Spam Detection

Imagine you want to build a model that predicts whether an incoming email is spam or not.

- Features (x):** You could use features like the presence of certain words (like "win" or "free"), the sender's email address, or the length of the email.
- Linear Function (Z):** You'd start by giving each of these features a weight based on how important they are in determining spam. For example, if the word "win" is very common in spam emails, it might get a high weight.
- Sigmoid Function and Probability:** The result is passed through the sigmoid function to get a probability. For example, an email containing the word "win" and from an unknown sender might result in a probability of 0.85 (or 85%) of being spam.
- Prediction:** If the probability is above 0.5, your model predicts that the email is spam; if it's below 0.5, it predicts it's not spam.

Logistic Regression is a way to predict if something belongs to one of two categories (like yes/no, spam/not spam, pass/fail) based on some input data. Importantly, it gives us a probability that something belongs to a certain category.

1. Input Feature Space (x):

- Think of the input features, x , as characteristics or attributes that describe each item you're trying to classify. For example, in email spam detection, x could represent the number of times specific words (like "win" or "free") appear in the email.

2. Linear Function (Z):

- The first step involves applying a linear function to the input features. This is a simple mathematical function that combines the features using weights to measure their importance. The equation for this is:

$$Z = \theta^T x + \theta_0$$

Here, θ represents the weights assigned to each feature (e.g., how important "win" is in determining spam), and θ_0 is a bias term.

3. Adding Nonlinearity (Sigmoid Function):

- After calculating the linear combination, we apply the sigmoid function, $g(z) = \frac{1}{1+e^{-z}}$, which squashes the result to a value between 0 and 1. This value is interpreted as the probability that the input belongs to a particular class (e.g., the likelihood that an email is spam).
- If $g(z) = 0.85$, it indicates an 85% chance that the email is spam.

4. Log-Likelihood Loss:

- The model evaluates how well it's performing by comparing its predicted probability with the actual classification (spam or not spam). The log-likelihood loss function helps measure the difference between these two. It is represented by:

$$L_{hl}(g^i, y^i) = y^i \log(g^i) + (1 - y^i) \log(1 - g^i)$$

This formula checks how far the prediction deviates from the actual result.

5. Optimization:

- To make the model better, we adjust the weights θ to reduce the errors or minimize the log-likelihood loss. Optimization techniques like gradient descent are used here. In addition, regularization (adding a penalty for large weights) can prevent the model from becoming too complex, which could cause overfitting.
- The cost function that incorporates regularization is:

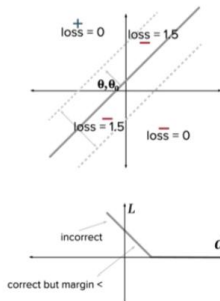
$$J(\theta, \theta_0) = \frac{1}{n} \sum_{i=1}^n L_{hl}(g^i, y^i) + \lambda \|\theta\|$$

6. Inference:

- Once the model is trained, we can use it to predict outcomes. If the output $g(z)$ is greater than 0.5, the model classifies the input as one class (e.g., spam). If $g(z)$ is less than or equal to 0.5, it classifies it as the other class (e.g., not spam).

The example of **email spam detection** illustrates this process, where features like specific words are weighted based on their contribution to the classification, and the sigmoid function helps produce a probability score to decide if an email is spam or not.

Support Vector Machines or SVMs



- As in logistic regression, we now optimize to minimize the loss (and to penalize large θ), but now with L_h rather than L_{hll} :

$$J(\theta, \theta_0) = \left(\frac{1}{n} \sum_{i=1}^n L_h(x^{(i)}, y^{(i)}) \right) + \lambda \|\theta\|$$

Inference: decide class = 1 if $d > 0$
else class = 0

Ex.
Handwriting
Recognition



SVMs are a powerful classification tool that works by finding the optimal boundary, or "hyperplane," that separates data points of different classes. The goal is to maximize the margin, which is the distance between the hyperplane and the closest data points from each class, known as "support vectors."

Key Components:

Loss Function:

- SVMs minimize a specific loss function to improve accuracy, aiming to maximize the margin between classes.
- Margin:** The distance between the decision boundary and the closest data points. A larger margin generally leads to better generalization.
- Misclassification and Margin Loss:**
 - Correctly classified points outside the margin contribute no loss.
 - Correctly classified points inside the margin have a small loss, encouraging the model to push points further from the boundary.
 - Misclassified points (on the wrong side of the hyperplane) contribute significant loss.
- Regularization:** Prevents overfitting by adding a penalty for model complexity.

Mathematical Expression:

- The equation combines two terms:
 - The first term $\left(\frac{1}{n} \sum_{i=1}^n L_h(x^{(i)}, y^{(i)})\right)$ is the average loss across all points, indicating classification performance.
 - The second term $\lambda \|\theta\|$ is the regularization term that controls model complexity, preventing overfitting.

Inference:

- After training, the SVM uses the learned hyperplane to classify new data points based on their distance from the hyperplane. The further a point is from the boundary, the more confident the classification.

Example: Handwriting Recognition

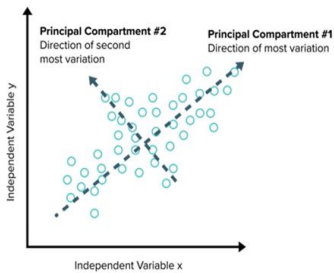
- **Training:** Given a dataset of images of handwritten digits (e.g., "0" and "1"), SVM analyzes the data and finds the best hyperplane that separates the two classes.
- **Loss Function in Action:**
 - If a "0" is far from the hyperplane on the correct side, the loss is 0.
 - If a "0" is close to the hyperplane but correctly classified, there's a small loss.
 - If a "0" is misclassified as a "1," the loss is significant.
- **Prediction:** For new, unseen images, the SVM calculates the distance from the hyperplane to classify the image confidently.

In summary, SVMs aim to find the boundary that best separates classes by maximizing the margin between them, helping ensure more confident and accurate predictions, as demonstrated in tasks like handwriting recognition.

Comparative Analysis of Classification and Regression Algorithms		
Feature	Classification	Regression
Output Type	Produces discrete, categorical labels (e.g., yes/no decisions, class labels). Suitable for datasets where the result is limited to specific categories.	Produces continuous numerical outputs (e.g., price, temperature). Ideal for predictions where the result can vary widely and is not limited to specific categories.
Common Algorithms	Logistic regression, support vector machines (SVM), decision trees, random forests, and naive Bayes. These algorithms are designed to handle data that can be easily segmented into distinct classes.	Linear regression, polynomial regression, and ridge/lasso regression. These methods are capable of modeling complex relationships by fitting a continuous curve to the data.
Evaluation Metrics	Focuses on classification accuracy, precision (positive predictive value), recall (sensitivity), F1 score, and the use of a confusion matrix to visualize the performance of the classifier across different classes. These metrics help in understanding the effectiveness of classification in correctly labeling each class.	Utilizes mean squared error (MSE), mean absolute error (MAE), and root mean squared error (RMSE) to measure the average error magnitude without considering direction. These metrics are crucial for evaluating how close the predicted values are to the actual values.
Use Cases	Widely used in applications where binary or multiclass sorting is required, such as email spam detection, medical diagnosis, and image classification. Classification is crucial in scenarios where discrete decisions need to be made.	Common in economics, finance, and other fields that require precise numerical forecasts, such as predicting stock prices, real estate values, and weather conditions. Regression fits naturally into contexts where the output is a quantity rather than a category.
Data Sensitivity	Generally more robust against outliers because the primary goal is to categorize inputs correctly rather than predict exact values. Misclassified points can often be handled through robust training methods or by adjusting decision thresholds.	More sensitive to outliers because the accuracy of the prediction depends heavily on the range and distribution of the dataset. Outliers can significantly skew the regression model, leading to poor performance on typical data.
Complexity	Depending on the complexity of the data and the number of classes, classification models can be straightforward (e.g., binary classification) or complex (multiclass problems). The complexity often influences the choice of algorithm and the feature engineering required.	Can involve complex data transformations and tuning, especially when dealing with non-linear relationships. Polynomial features, normalization, and regularization are common techniques used to improve regression model performance and handle overfitting.

Conclusion

The choice between classification and regression is determined by the nature of the output variable (categorical vs. continuous) and the specific requirements of the application. Understanding these differences is essential for selecting the right algorithm and achieving high-quality predictive performance in various domains.



Source: an, Originally published by Packt_Pub, "Principal Component Analysis: Unsupervised Learning Models" Hacker Noon, January 26, 2021, <https://hackernoon.com/principal-component-analysis-unsupervised-learning-models-8f8c7d3d2>.

Clustering

- **Unsupervised Training:** have only input vectors x , but no associated labels y
- **Goal:** identify "natural" groupings of the data
- **Inference:** for previous or new data point x : decide cluster to which it belongs



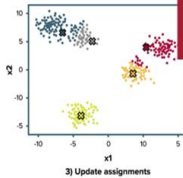
Characteristics of clustering:

- Useful as part of a larger machine learning solution in combination with other machine learning methods
- Help determine representative labels for just few examples



Clustering: k-means Algorithm

- Assume $m_1, \dots, m_k$ initial cluster means
- Use these means to assign each data point to the nearest cluster
- Adjust means m_i based on those memberships
- Iterate until some stopping criteria



Challenges in basic k -means:

- Assume k beforehand
- Additional approaches might be needed to sweep across different values of k

Ex. Customer Segmentation

Clustering Overview:

- **Principal Components Plot (Top Left):** The plot shows data points distributed along two principal components (Principal Component #1 and Principal Component #2), which represent directions of maximum variance in the data. These directions are like the most important patterns or trends in the data. This kind of plot is typically used in Principal Component Analysis (PCA) to reduce dimensionality before clustering, making it easier to spot groups or patterns in the data by focusing on these key trends.
- **Unsupervised Training:** Clustering is a type of unsupervised learning where the algorithm only has input vectors x and no associated labels y . The goal is to identify natural groupings in the data, which can help in making inferences about which cluster a new data point might belong to.
- **Characteristics of Clustering:** Clustering can be part of a larger machine learning solution, often used in combination with other methods. It can also help identify representative labels for a few examples in a dataset.

k-means Algorithm:

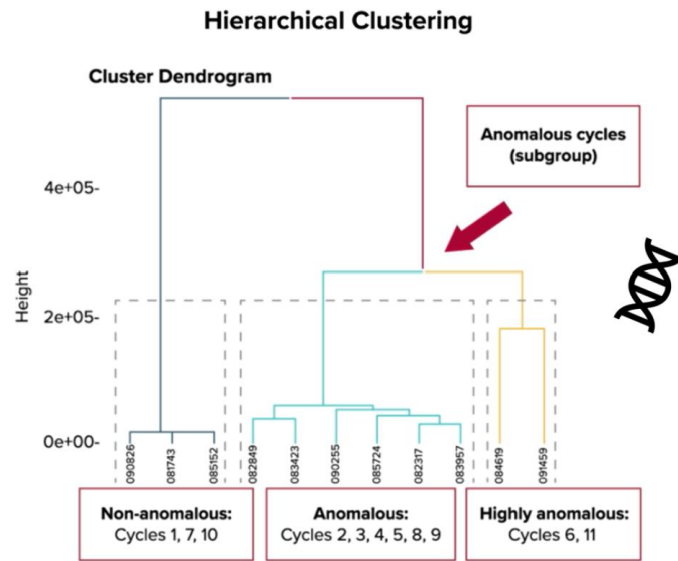
- **Steps Involved:**
 - Assume initial cluster centers (means).
 - Assign each data point to the nearest cluster center.
 - Adjust the cluster centers based on the points assigned to them.
 - Iterate until some stopping criteria are met (e.g., when cluster centers stabilize).
- **Challenges:** The basic k-means algorithm requires specifying the number of clusters k beforehand. In real-world applications, it may be necessary to test different values of k to find the optimal number of clusters.

Real-World Example of Clustering:

Imagine a retail company wanting to segment its customers based on purchasing behavior. They could use clustering to group customers who have similar shopping patterns. For example:

- **Cluster 1:** Frequent shoppers who spend small amounts.
- **Cluster 2:** Infrequent shoppers who make large purchases.
- **Cluster 3:** Shoppers who are highly responsive to promotions.

By identifying these clusters, the company can tailor its marketing strategies to target each group more effectively. For instance, they could offer special discounts to frequent small spenders to encourage larger purchases, or send exclusive promotions to the infrequent large spenders to increase their shopping frequency.



**Ex. Customer Service
Ticket Optimization**

A **hierarchical clustering dendrogram** is used in data analysis to visualize the arrangement of clusters formed by a hierarchical clustering algorithm.

Explanation of the Diagram

1. Dendrogram Structure:

- The diagram is a tree-like structure, where each branch represents a cluster of data points (in this case, possibly gene expression cycles or some other biological data).
- The **height** of the branches (the y-axis) represents the distance or dissimilarity between the clusters being joined. The greater the height, the more dissimilar the clusters are.

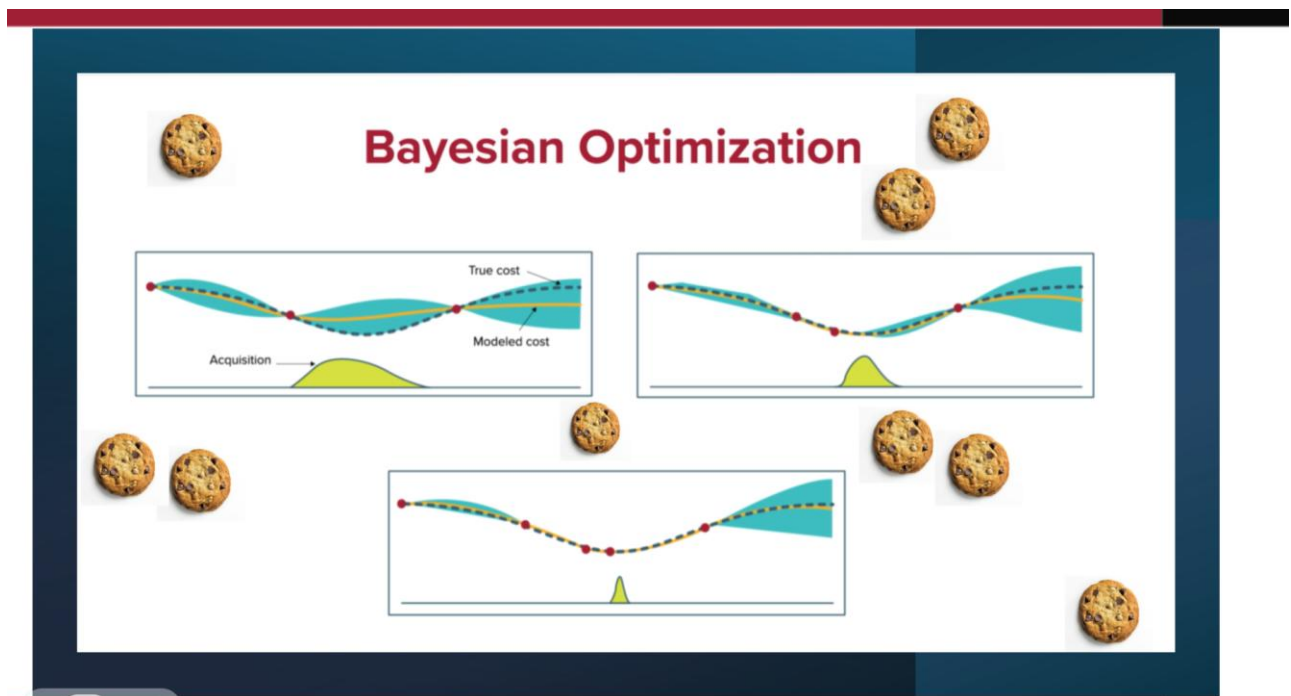
2. Clusters:

- The diagram shows three main clusters:
 - **Non-anomalous cycles:** These are the data points that fall within expected ranges or normal behavior. In the diagram, they are represented as a cluster on the left, labeled "Cycles 1, 7, 10."
 - **Anomalous cycles:** This cluster contains data points that deviate from normal but are not extreme. In the diagram, this is the central cluster labeled "Cycles 2, 3, 4, 5, 8, 9."
 - **Highly anomalous cycles:** These data points are the most different from the norm, indicating significant deviation. This is the cluster on the right, labeled "Cycles 6, 11."

3. Interpretation:

- The dendrogram suggests that the data can be naturally divided into groups based on their similarity, with some groups showing normal behavior, some being slightly anomalous, and others being highly anomalous.

A simple use case for hierarchical clustering in customer service could be categorizing support tickets. For example, a company can use hierarchical clustering to group customer support tickets based on the similarity of the issues reported. Tickets that describe similar problems, like login issues or payment processing errors, would be grouped together. This allows the customer service team to identify common issues more quickly and prioritize resources to resolve the most frequent problems efficiently, leading to faster response times and improved customer satisfaction.



Bayesian optimization is a method used to optimize an objective function, particularly when the function is expensive to evaluate.

Overview of Bayesian Optimization:

Initial Observations:

- The process begins with a few known data points (shown as red dots) that represent observations of the objective function we're trying to optimize.

Modeling the Objective Function:

- A probabilistic model (typically a Gaussian process) is used to represent our understanding of the objective function based on these observations. The model not only predicts the function's value (the "modeled cost") but also quantifies the uncertainty of this prediction, depicted by the light blue intervals.

Uncertainty and Exploration:

- As the distance from known data points increases, the uncertainty about the function's value also increases. This is shown by the widening of the light blue intervals.
- The goal is to find new points (data points) that will best reduce this uncertainty.

Acquisition Function:

- An acquisition function is used to decide where to sample next. It balances the need to improve the model by exploring uncertain areas and the desire to find the minimum of the objective function.
- The acquisition function suggests the next point to evaluate, typically where there is both high uncertainty and a good chance of finding a lower objective function value.

Iterative Improvement:

- Over successive iterations, as new data points are added, the model improves, and the estimated minimum of the objective function becomes more accurate.

This method is particularly useful in scenarios where evaluating the objective function is costly or time-consuming, as it guides the search in an efficient and informed manner. The ultimate goal is to converge on the optimal value of the function with a high level of confidence.

Example: Optimizing a Recipe for the Best Chocolate Chip Cookies

Imagine you're a baker trying to create the perfect chocolate chip cookie. You have a recipe, but you're not sure about the best amount of sugar to use to make them taste just right. You start by baking three batches with different amounts of sugar: low, medium, and high.

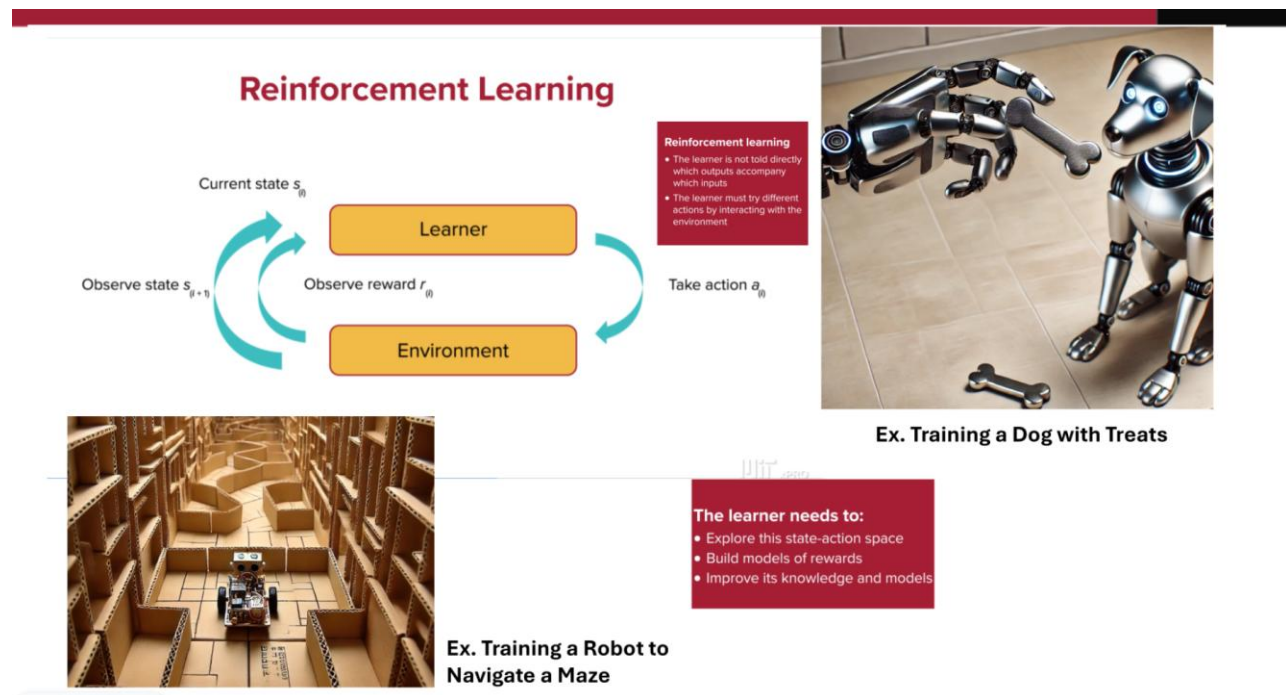
These batches represent your initial data points (the red dots in the diagram). You taste the cookies and find that medium sugar is pretty good, but you're not sure if it's the best because you only tried three amounts.

Now, instead of randomly trying another sugar amount, you use Bayesian optimization to help decide the best next batch to bake. This optimization method uses what it knows from your first three batches and predicts how different sugar amounts might affect the taste of the cookies. The "modeled cost" in the diagram represents how tasty the cookies are expected to be with different sugar levels.

But there's uncertainty (the light blue area of the diagram) because you haven't tried every possible amount of sugar yet. You might try baking with a sugar level you haven't tested yet, one where the uncertainty is high but could result in the best cookie.

The acquisition function helps you pick the next sugar amount to test, balancing between trying something completely new (which could improve your recipe) and refining what you already know (to make sure you're on the right track). As you keep baking and tasting, each new batch of cookies brings you closer to finding the perfect amount of sugar, the optimum recipe.

Over time, you'll bake enough batches that you become confident about the perfect sugar amount, resulting in the best possible chocolate chip cookies!



Reinforcement learning is a powerful method in machine learning where an algorithm, referred to as the "learner," learns by interacting with its environment. Unlike supervised learning, where the learner is given direct feedback on what outputs are correct for specific inputs, reinforcement learning requires the learner to figure this out through trial and error.

Here's a step-by-step breakdown of how it works:

Observation of the Current State: The learner first observes the current state of the environment. This state could be deterministic or probabilistic, meaning it could be certain or uncertain.

Taking an Action: Based on this observation, the learner takes an action within the environment. This action is chosen with the aim of achieving the best possible outcome.

Receiving a Reward: After taking the action, the learner receives feedback in the form of a reward. This reward could be immediate or deferred, depending on how the action influences the state of the environment.

State Transition: The action taken causes the environment to transition to a new state. The learner then observes this new state, and the cycle repeats.

The goal of reinforcement learning is for the learner to identify which actions lead to the best long-term rewards, not just immediate rewards. Over time, by exploring different actions and their outcomes, the learner builds a model of how to act in different states to maximize rewards.

Example: Imagine training a dog to sit. The environment is your living room, the dog is the learner, and the action is sitting. When the dog sits on command, you give it a treat (the reward). Initially, the dog may try different actions, lying down, standing, barking; before figuring out that sitting earns the treat. Over time, the dog learns that sitting is the best action to take when you give the command because it leads to the reward. The dog doesn't just learn to sit for the treat but eventually understands to sit on command for a variety of rewards, even if the treat isn't immediate.

Reinforcement learning can be applied similarly in AI, such as training a robot to navigate a maze. The robot (learner) tries different paths (actions) and learns which path leads to the exit (long-term reward), even if some paths have temporary obstacles (short-term penalties).